Operadores: calcular, comparar, asignar

Lección 4 — 1MAK, SST-INF-01c — Operadores matemáticos, de comparación, de asignación

02/10/2026

1 Repaso: los tres tipos de contenedor

Nota

La vez pasada vimos tres tipos de **variables** (*variabili*), es decir, tres tipos de **contenedores** donde poner datos:

- `char`: de **8 bits**, es decir un byte; contiene números desde un mínimo de **0** hasta un máximo de **255**;
- `int`: mínimo **16 bits**, es decir mínimo 2 bytes, pero depende de la **arquitectura de la máquina**. En las computadoras grandes, como las del laboratorio, que trabajan a **64 bits**, el entero toma la arquitectura de la máquina. A nosotros nos interesan los `int` de 16 bits, porque vamos a trabajar con **Arduino**, que usa `int` de 16 bits. Con una caja de 16 bits se guardan números de -32.768 **a** $+32.767$;
- `float`: caja de **32 bits**, es decir 4 bytes, para los **números con coma**.

Todo está en la **guía** de la clase anterior. La **tabla** escrita en el pizarrón:

Tipo	Bits	Bytes	Rango dicho en el aula
`char`	8	1	$0 \to 255$
`int`	16	2	$-32.768 \to +32.767$
`float`	32	4	$-2\,\text{G} \to +2\,\text{G} \quad (-2 \cdot 10^9 \to +2 \cdot 10^9)$

Para no escribir 9 ceros se usa la **notación con letras** (“2 giga”, 2G) o la **notación científica** ($2 \cdot 10^9$), que hay que conocer porque también la están viendo en matemática. Cuidado con las **dos correcciones** de abajo.

Nota

El rango del `char` en este curso. En este curso, y por lo tanto también en las pruebas, para el `char` se usa el **rango de 0 a 255**. Para que sepas más, hay **tres niveles** distintos. El **estándar del lenguaje C** **no establece** si el `char` tiene signo: **deja la decisión** a quien hace el compilador. El **compilador** decide para la **plataforma** para la que está hecho (y por lo general la elección se puede cambiar con una opción). Para **Arduino**, la **documentación oficial** indica para el `char` el rango de -128 a $+127$, es decir **con signo**. El rango de 0 a 255 es siempre el del tipo **sin signo**, **`unsigned char`** (en Arduino también se llama `byte`). En los dos casos las **combinaciones** son 256.

Atención

Corrección sobre el rango del `float`. En el aula se dijo que el `float` va "más o menos de menos **2 mil millones** a más 2 mil millones" (es decir de $-2 \cdot 10^9$ a $+2 \cdot 10^9$, en **notación científica**). Ese es el **rango** de un **entero** de 32 bits. El `float` de 32 bits llega **mucho más lejos**: más o menos $\pm 3{,}4 \cdot 10^{38}$, porque divide sus bits entre signo, **exponente y mantisa**, como la notación científica. A cambio tiene una precisión limitada, más o menos **7 cifras significativas**. El detalle ya está en la guía A03.

Nota

¿Firmware o electrónica? Esta es una **clase de firmware**. Según el calendario, el **lunes** se hace **electrónica**, el **martes y el viernes** firmware, para poder avanzar **en paralelo** con el programa. En primer año, electrónica y firmware son dos cosas distintas; en segundo empiezan a unirse; desde tercero en adelante van a ser prácticamente **una sola cosa**, porque no se puede hablar de electrónica sin hablar de firmware, y al revés.

2 Qué son los operadores

Definición

Ahora que sabemos crear variables, tenemos que hacerlas **interactuar** entre sí: dos contenedores que se quedan ahí solos **no sirven para nada**. Hay que combinarlos, es decir, aplicar **operaciones** que den resultados. Al fin y al cabo, la computadora y el Arduino son máquinas electrónicas que hacen una sola cosa: **cálculos**. Las operaciones las hace la máquina, pero somos nosotros los que tenemos que **explicarle** cómo hacerlas.
Los **operadores** (*operatori*) son los símbolos que permiten **combinar** entre sí varios contenedores (o varios números).

Ejemplo

Si tomo los números 3 y 2 y los combino con el operador +, el resultado es **5**, porque es el **operador suma**. Si en lugar del más uso el menos, el resultado es distinto: **1**. Mismos números, **operador distinto**, **resultado distinto**.

3 Los operadores matemáticos

Definición

Los **operadores matemáticos** (*operatori matematici*) son los que estudiamos en la primaria, más uno nuevo. Son **cinco**:

Símbolo	Operación
+	suma (*somma*)
-	resta (*sottrazione*)
*	multiplicación (*moltiplicazione*, asterisco)
/	división (*divisione*, se llama **slash**, barra)
%	**módulo** (*modulo*): da el resto de la división entre enteros

Ejemplo

Si en un programa escribo `8 / 2`, quiere decir 8 dividido 2: el resultado es 4. Si escribo `5 * 3`, es la **multiplicación**: da 15.

Atención

Usen los símbolos que la máquina espera. Un alumno hizo notar que en matemática la multiplicación también se escribe con el **puntito**. En matemática sí; en programación el puntito no se puede usar, hace falta el **asterisco**. No pueden escribir `5.3` y esperar que la máquina entienda que tiene que **multiplicar**: para ella es el número 5,3. La máquina **multiplica solo** cuando encuentra el asterisco. Tampoco sirve la "x". Para la división no se usan los **dos puntos** ni el símbolo $\div$, sino la **barra** (*slash*) `/`, que en el teclado italiano está en la tecla del 7 (`Shift + 7`).

3.1 El operador módulo

Definición

El símbolo `%` normalmente lo usan para el porcentaje. En programación **no tiene nada que ver** con el **porcentaje**: es más, el cálculo del porcentaje como operador **no existe**, hay que hacer la cuenta **a mano**. Este símbolo se llama **módulo** (*modulo*) y da el **resto de la división** entre enteros.

Ejemplo

Los ejemplos escritos en el pizarrón.

- ¿Cuánto da $5 \div 2$? En matemática, 2 y medio. Pero solo con **números enteros**, es decir sin coma, da **2**, con resto **1**: $2 \times 2 = 4$, para llegar a 5 falta 1. En programación `5 / 2` da justamente `2`, y `5 % 2` da `1`.
- $11 \div 3$: 3 por 3 da 9, entonces da **3** con **resto 2**. `11 / 3` da `3`, `11 % 3` da `2`.
- $28 \div 5$: 5 por 5 da 25, entonces da **5** con **resto 3**. `28 / 5` da `5`, `28 % 5` da `3`.

Atención

El módulo **no** da el resultado de la división: da **solo el resto**. La barra **no** da el resto: da solo el **resultado entero**. Si necesitan los dos, tienen que escribir **dos líneas de código**: una con la división y otra con el módulo. La máquina **no lo hace automáticamente**.

```
int quoziente;
int resto;
// quoziente (cociente) vale 5
quoziente = 28 / 5;
// resto vale 3
resto = 28 % 5;
```

El módulo muchas veces genera un poco de **confusión**: por eso lo repetimos varias veces. Va a ser muy útil más adelante, por ejemplo para saber si un número es **par** o **impar**: un número es par si `numero % 2` **vale 0**.

Atención

La **división entre enteros** **pierde la parte decimal**. La computadora **no redondea**: **corta** todo lo que está después de la coma. `5 / 2` da 2, no 3. Para tener el resultado con coma hacen falta variables **`float`**.

4 Los operadores de comparación

Definición

Los operadores matemáticos dan **siempre un resultado numérico**: $6+2$ da 8, $9-5$ da 4. Es bastante obvio, pero nunca se había dicho explícitamente. No todos los operadores funcionan así: existe una categoría de operadores que, en lugar de dar como resultado **un número**, da como resultado una **condición de verdad**, es decir **verdadero o falso** (*vero o falso*). Son los **operadores de comparación** (*operatori di confronto*). Son **seis**:

En C	En matemática	Pregunta
`>`	$>$	¿el primero es mayor que el segundo?
`<`	$<$	¿el primero es menor que el segundo?
`>=`	$\geq$	¿el primero es mayor **o igual**?
`<=`	$\leq$	¿el primero es menor **o igual**?
`==`	$=$	¿son iguales? (**igualdad**, *uguaglianza*)
`!=`	$\neq$	¿son distintos? (**desigualdad**, *disuguaglianza*)

Ejemplo

Ejemplo en el pizarrón.

```
int a;
int b;
a = 5;
b = 7;
a < b;
a > b;
```

¿`a < b` puede dar un resultado numérico? No: solo puede dar **verdadero o falso**. Entonces, ¿`a` es menor que `b`? En el aula alguien respondió que sí "porque la a viene antes que la b". Cuidado: acá no estamos preguntando si la letra `a` viene antes que la `b` en el alfabeto. `a` es una **variable** que contiene 5, `b` contiene 7: la pregunta es **¿5 es menor que 7?** La comparación se

hace siempre sobre el **contenido** de las variables. Entonces `a < b` es **verdadero**, mientras que `a > b` es **falso**.

Ejemplo

¿Y si `b` vale 5? En el pizarrón el profesor cambia el valor de `b`:

```
int a;
int b;
a = 5;
b = 5;
a < b;
a > b;
a <= b;
```

¿`a` es menor que `b`? **No**: falso, 5 no es menor que 5. ¿`a` es mayor que `b`? **No**: también es falso. ¿Y `a` es menor **o por lo menos igual** a `b`? **Sí**: verdadero, porque la igualdad existe. De acá viene la necesidad de los símbolos "o igual".
Durante estas preguntas un alumno respondió "los vi, pero no me los acuerdo": no es un problema, pero hay que superarlo, tomándose el tiempo en casa, la semana siguiente, para **repasar** estos símbolos.

Definición

Mayor o igual, menor o igual. En el teclado **no existe** el símbolo $\leq$ de la matemática. En C se escribe **combinando dos símbolos**: >= equivale a $\geq$, <= equivale a $\leq$. Agregar el igual cambia la respuesta en los **valores límite**: `5 < 5` es **falso** (5 no es menor que 5), pero `5 <= 5` es **verdadero**, porque por lo menos la igualdad existe.

5 El operador de asignación

Definición

La última categoría es la que genera **más confusión** en los alumnos de cualquier escuela, de cualquier nivel: por eso hay que verla **con calma** y bien. Si en clase el profesor de matemática escribe $a = b + 5$, está escribiendo una **ecuación**: se pregunta para qué valores de a es verdadera, es decir, **compara** lo que está a la izquierda con lo que está a la derecha. Es un símbolo de igualdad.
En C el símbolo = **no es la igualdad**. Es el **operador de asignación** (*operatore di assegnazione*): **copia el valor** que está a su derecha en la **variable** que está a su izquierda. No se está preguntando si son iguales: se está **moviendo una información** de un lugar a otro. Por eso la **asignación** nunca da un resultado verdadero o falso.

Ejemplo

¿Cuánto valen `a` y `b` al final?

```
int a;
```

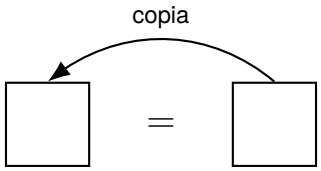
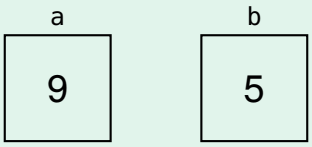


Figura 1: El esquema dibujado en el pizarrón: el valor de la caja de la derecha se copia en la de la izquierda.

```
int b;
b = 9;
a = b;
b = 5;
```

En el aula llegaron respuestas de todo tipo: “a vale 14 porque 9 más 5”, “b vale 9 y a vale 11”, “b vale 7... no, 9... b era 5”. Muchos alumnos respondieron “b 9 y a 5”, es decir con los valores **intercambiados**, y uno dijo “a es igual a b, son dos b”, como si las dos variables quedaran unidas. Al final la respuesta correcta la dio un alumno. ¿Dónde estaría la suma? Para hacer la **suma** hace falta el símbolo +, y acá no está. ¿Por qué justo la suma y no la multiplicación o la división? La máquina **no inventa nada**: hace solo lo que está escrito.
Leamos el código una línea a la vez, de arriba hacia abajo:

1. **creo** un contenedor que se llama a;
2. **creo** un contenedor que se llama b;
3. en el contenedor b **pongo** el número 9;
4. en el contenedor a pongo el **mismo contenido** de b: en este momento b vale 9, entonces en a va 9;
5. en el contenedor b pongo 5: el 9 **desaparece** y en su lugar entra el 5.



Resultado: **a vale 9** y **b vale 5**, como en las dos cajas dibujadas en el pizarrón (no “b 9 y a 5”). Asignar dos veces b no es un problema: cuenta siempre **la última asignación**. Y si b cambia después, a **no cambia** (es el error de quien decía “son dos b”): a = b **no crea una unión** entre las dos variables. Solo dice “toma el contenido de b ahora y cópialo en a”, y nada más.

Nota

En el aula el profesor insistió para que todos respondieran: “quiero interactuar, no se puede dar una clase como si fuera un discurso de funeral” (en italiano: *“un’orazione da cadavere”*). Una **respuesta equivocada** no es un problema: se corrige. El problema es **no intentarlo**.

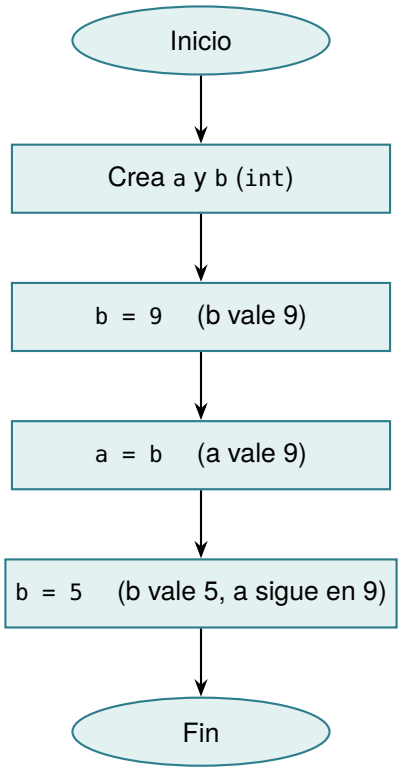


Figura 2: Diagrama de flujo del ejemplo: las instrucciones se ejecutan una a la vez, de arriba hacia abajo.

6 ¿Igual simple o doble igual?

Nota

Pregunta de un alumno: "¿y los dos iguales?". Respuesta: eso es la **comparación**. En el pizarrón, en la columna de los **operadores de comparación**, el doble igual primero se había escrito con la etiqueta "Comparación" (*Confronto*), y después se corrigió en "**Igualdad**" (*Uguaglianza*), con abajo la "**Desigualdad**" (*Disuguaglianza*) `!=`.

Atención

El **error más común** de quien empieza a programar: confundir = con ==.

- = **simple** sirve para **asignar** un dato, es decir, para copiarlo de un lugar a otro. Es una **acción**.
- == **doble** sirve para **comparar**: pregunta si dos valores son iguales y responde verdadero o falso, sin modificar nada. Es una **pregunta**.

Pregúntense siempre: cuando tengo que copiar un dato, decidir cuánto vale algo, necesito **el igual**; cuando me estoy preguntando si dos valores son iguales, tengo que usar el **doble igual**. Es importante, porque si no sus códigos **no van a funcionar nunca**. Cuidado también con escribir `5 = a`: a la izquierda de la asignación tiene que haber **siempre una variable**, porque **un número no puede contener** otro valor.

Ejemplo

Ejemplo en el pizarrón.

```
int a;
int b;
a = 5;
b = 7;
a = b;
a == b;
```

`a = b` es una **asignación**: tomo el contenido de `b` y lo copio en `a`. Ahora `a` vale 7 y `b` vale 7. Después `a == b` **pregunta si son iguales**: ¿7 es igual a 7? **Verdadero**. Si quitara la línea `a = b;`, la comparación sería **falsa**, porque preguntaría si 5 es igual a 7. Una cosa es **copiar** el dato, y otra cosa es **comparar** dos variables o dos números.

Ejemplo

La desigualdad. Con `a = 5` y `b = 7` (sin la línea `a = b`), escrito en el pizarrón:

```
a != b;
a == b;
```

`a != b` pregunta "¿`a` es distinto de `b`?": la **pregunta contraria** a la igualdad. Es **verdadero**, porque 5 y 7 son **números distintos**. `a == b` en cambio es **falso**. Se lee "des-i-gual-dad" (en italiano "di-su-gua-glian-za"): **signo de exclamación** seguido de igual.

7 Ejercicios en el pizarrón: ¿verdadero o falso?

Ejemplo

Ejercicio 1.

```
int a;
a = 11;
int b;
b = 3;
(a % b) == 2;
```

En el pizarrón, antes del ejercicio está escrito "V o F $\rightarrow$" (*vero o falso*: ¿verdadero o falso?). ¿Es verdadero o falso? Primero se calcula lo que está **entre paréntesis** (en el pizarrón el profesor puso abajo una llave con un 2 escrito): `a % b` es el **resto** de $11 \div 3$, es decir **2** (no la división, que daría 3). Después la pregunta: ¿2 es igual a 2? **Verdadero**.

Ejemplo

Ejercicio 2.

```
int x;
int y;
int z;
x = 5;
```

```
y = 3;
z = x * y;
z > 10;
```

¿Es verdadero o falso? `z` vale `x` por `y`, es decir $5 \times 3 = 15$: **el asterisco** es la multiplicación. ¿15 es mayor que 10? **Verdadero**.

8 Resumen de los operadores vistos

Nota

El pizarrón al final de la clase:

- **5 operadores matemáticos**: suma `+`, resta `-`, multiplicación `*`, división `/`, módulo `%` (resto). Dan siempre un **número**.
- **6 operadores de comparación**: mayor `>`, menor `<`, mayor o igual `>=`, menor o igual `<=`, igualdad `==`, desigualdad `!=`. Dan siempre **verdadero o falso**.
- **1 operador de asignación**: `=`, que copia un dato de un lugar a otro (de derecha a izquierda).

Con estos tenemos **las bases** para empezar a hacer algo. La próxima vez se prende la computadora y empezamos con algo **más práctico**.

9 La primera prueba de firmware

Nota

El profesor le preguntó a la clase si prefería **pocas pruebas con muchos temas** para estudiar o **muchas pruebas con pocos temas**. Después se acordó la **primera prueba** (*verifica*) **de firmware**: **martes 6 de octubre**, más o menos una hora (había tres semanas para estudiar **tres páginas** de guías). La alternativa era el viernes siguiente, pero la clase prefirió el martes. El profesor sigue adelante con el programa y no hace un repaso especial: el **repaso** les toca a los alumnos. A la pregunta sobre la nota "de 0 a 100" según el número de preguntas, la respuesta fue que en una prueba de conocimientos las preguntas son de **reconocimiento**. La lista escrita en el pizarrón: **Binario $\leftrightarrow$ Decimal, Tipos de variables, Operadores**. Es una prueba **de conocimientos** (*conoscitiva*), no **práctica** (*operativa*), porque todavía no aprendimos a "hacer": preguntas de **opción múltiple**, del tipo "¿qué es este símbolo?" (es el menor o igual), "¿qué hace el módulo?" (**calcula el resto**), "¿qué hace el símbolo igual?" (copia el valor que está a su derecha en la variable que está a su izquierda). Temas:

- conversión de **binario a decimal** y al revés;
- **tipos de variables**: ¿qué números puede contener un **`int`**? (de -32.768 a $+32.767$); para un número con coma, ¿se usa `int` o `float`?;
- los **operadores** de esta clase.

Atención

Guías y pruebas. El profesor se había olvidado de publicar la **guía** (*dispensa*) de los operadores (esta): quien no tomó bien los apuntes la encuentra acá. Si algo está en la guía, **puede estar en la prueba**, aunque no se haya dicho en voz alta. Normalmente las guías contienen solo lo que se dice en clase, pero pueden agregar algo que se escapó al hablar: **no se basen** solo en sus apuntes, lean **toda la guía**. Si en la guía hubiera un error y ustedes se equivocaran por su culpa, **no los penalizarían**: de todos modos, las guías se revisan.

10 Qué recordar de esta lección

- Los **operadores** combinan variables y números. En C se usan solo los **símbolos** que la máquina espera: asterisco para multiplicar, barra (slash) para dividir.
- La **división entre enteros** da solo el resultado entero (`28 / 5` da 5); el **módulo** `%` da solo el resto (`28 % 5` da 3). Para tener los dos hacen falta dos líneas.
- Los operadores matemáticos dan un **número**; los operadores de comparación dan **verdadero o falso**.
- La comparación se hace sobre el **contenido** de las variables, no sobre su **nombre**.
- `>=` y `<=` se escriben con dos símbolos e incluyen el **valor límite**.
- `=` **asigna** (copia de derecha a izquierda), `==` **compara**. No confundirlos **nunca**.
- El código se lee **una línea a la vez**, de arriba hacia abajo: cuenta **la última asignación**, y `a = b` **no une** las dos variables para siempre.

11 Glosario: palabras en italiano que se usan en clase

Italiano	Español
operatore	operador
somma / sottrazione	suma / resta
moltiplicazione / divisione	multiplicación / división
asterisco	asterisco
slash	barra
modulo	módulo (resto de la división)
resto	resto
quoziente	cociente
pari / dispari	par / impar
confronto	comparación
maggiore / minore	mayor / menor
maggiore o uguale / minore o uguale	mayor o igual / menor o igual
uguaglianza / disuguaglianza	igualdad / desigualdad
vero / falso	verdadero / falso
assegnazione	asignación
contenitore	contenedor
dispensa	guía (apuntes del profesor)
verifica	prueba / examen
lavagna	pizarrón